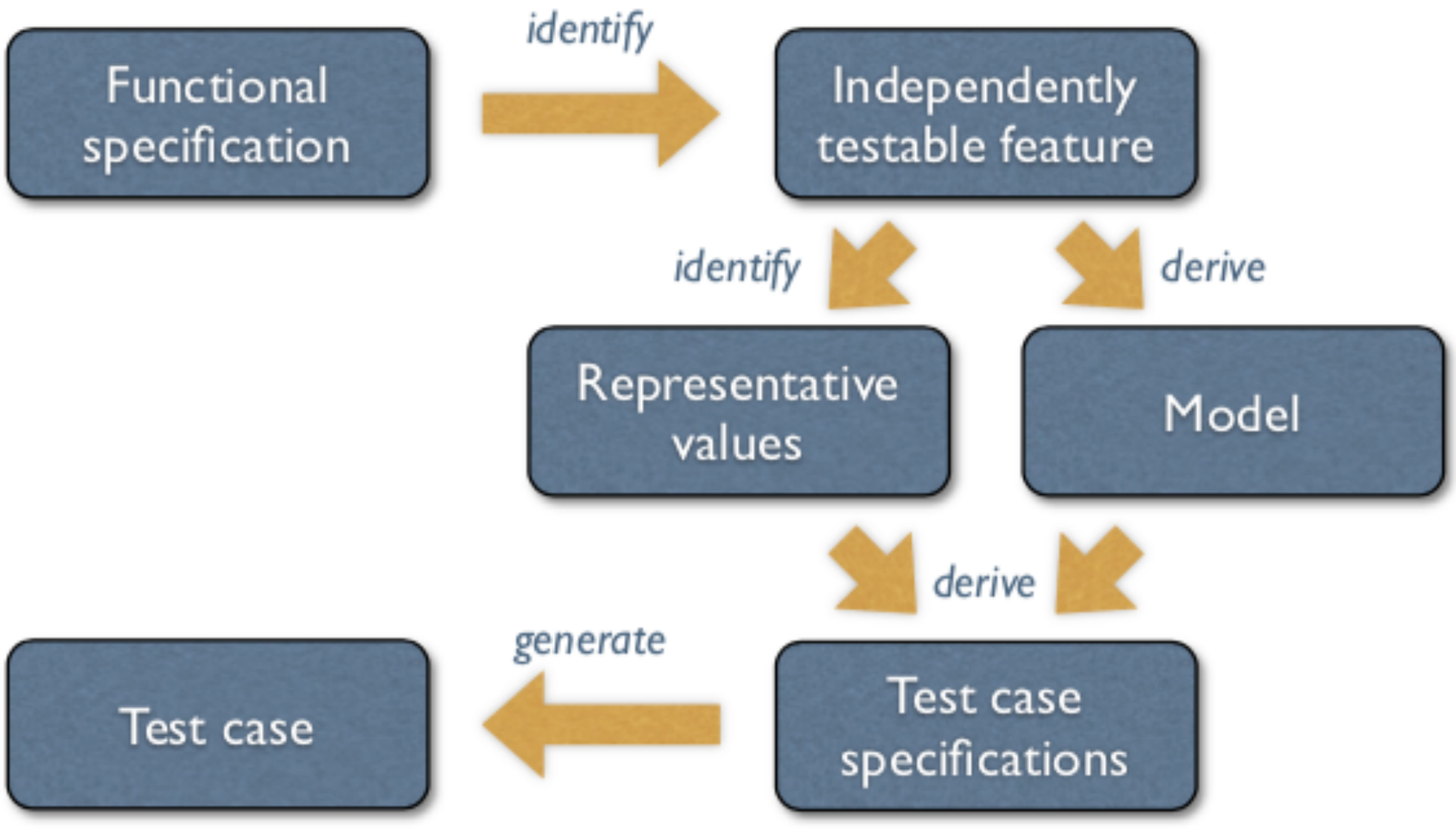
TESTE FUNCIONAL

Black-box testing

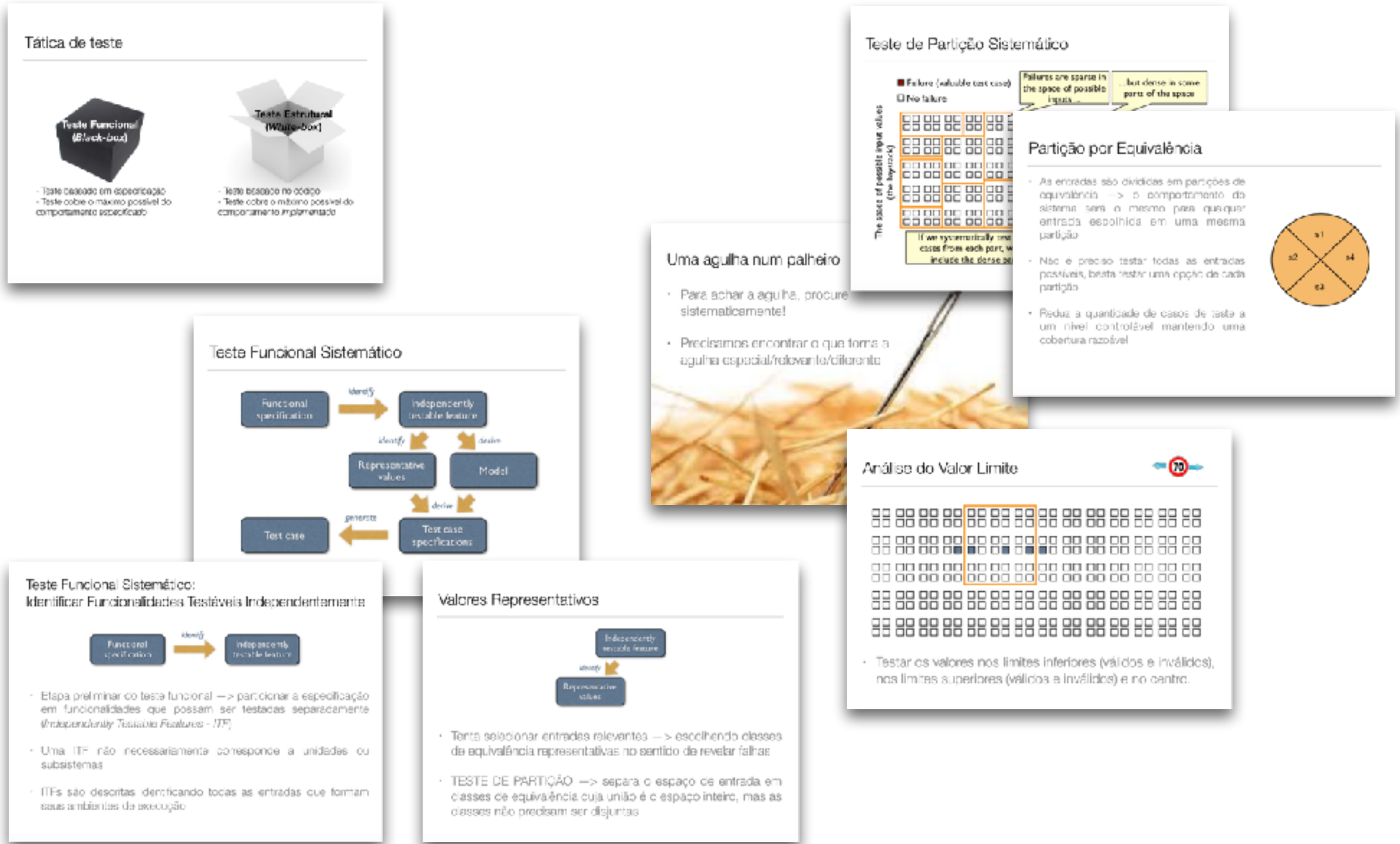
Tópicos Especiais em Engenharia de Software

Sabrina Souto

Teste Funcional Sistemático



Relembrando a aula passada...



Especificação

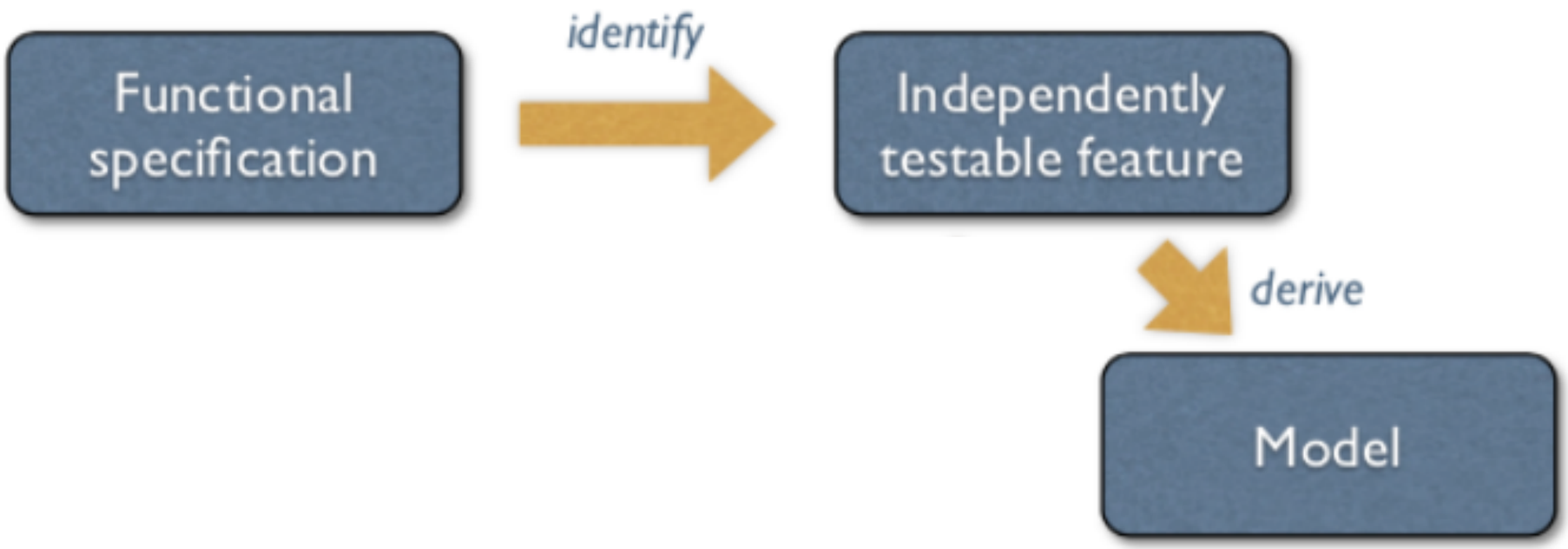
- Documento que representa o **comportamento** e as **características** que um sistema deve possuir

- Diversas formas:

  - Textual (linguagem natural) —> imprecisões/ inconsistências

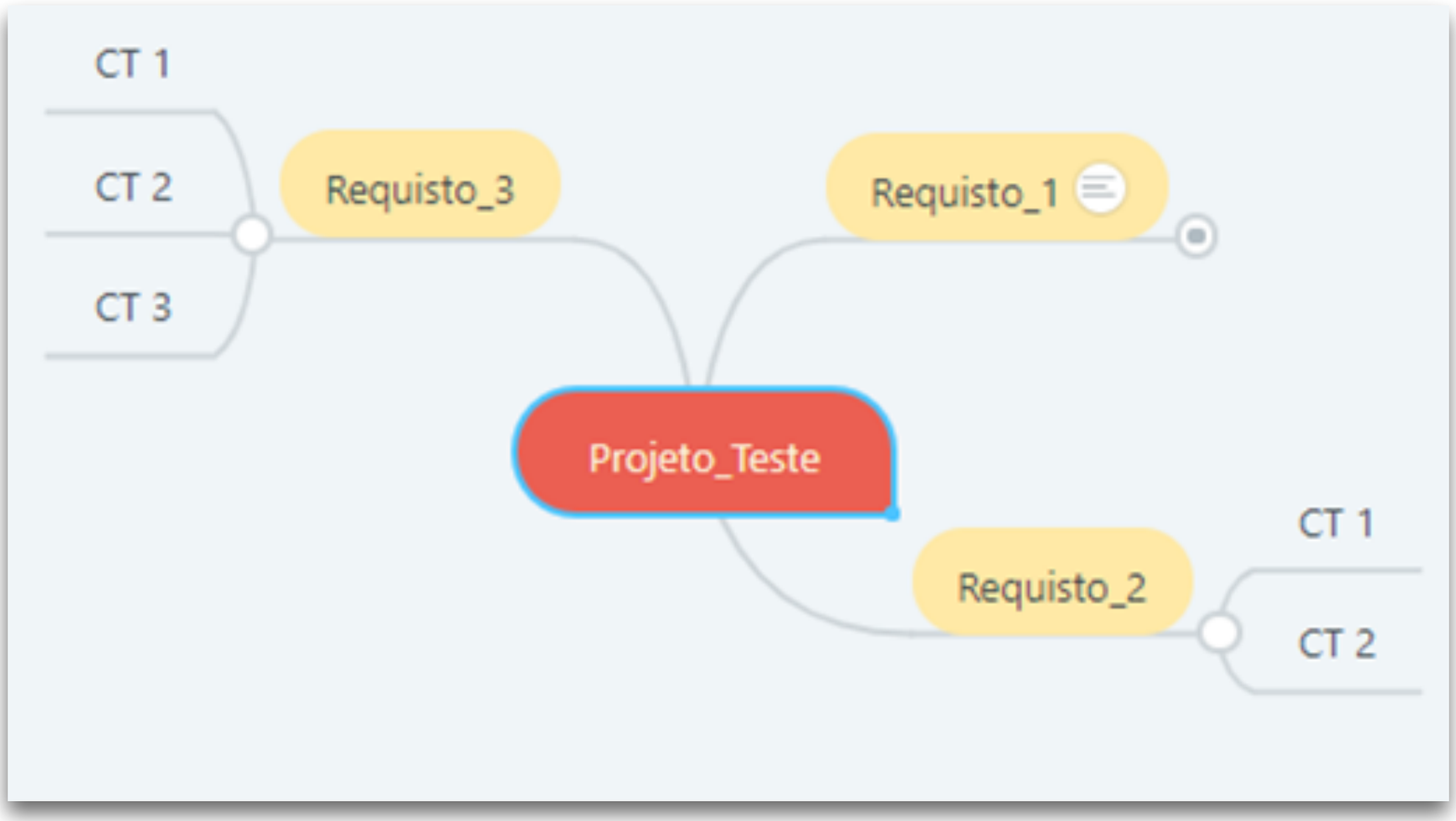
  - Especificação formal —> deve ser possível analisar algoritmicamente

Teste Baseado em Modelo



- Boa parte da atividade de teste é gasta buscando-se identificar o que o sistema deveria fazer
  - O resultado está correto? —> Qual seria o resultado correto?
- Um modelo é muito importante nessa tarefa!!

Mapas Mentais também são modelos! Porém, mais abstratos.



Teste Baseado em Modelo

- Durante a realização dos testes, um grande obstáculo é determinar exatamente quais são os objetivos de teste: **quais itens serão testados e como**

- **Especificação** - exemplo retirado de um doc. requisitos —> “Se um dígito inválido é fornecido, ele deve ser tratado de maneira adequada”

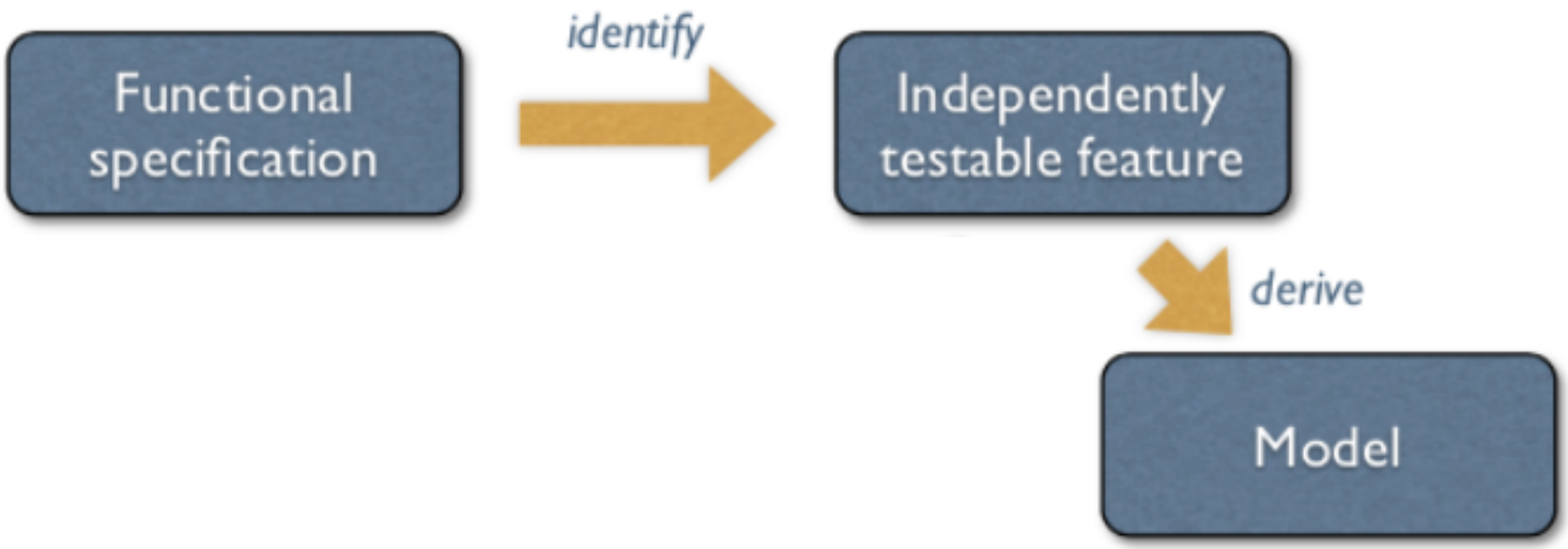
- Os testadores sempre constroem modelos, ainda que de maneira informal

- Mas qual o melhor formato de modelo?

Teste Baseado em Modelo

- Mas qual o melhor formato de modelo?
- Um modelo que está só na cabeça do testador dificulta a automatização dos testes
- Para escrever um script de teste ou um plano de teste, o testador deve entender os passos necessários para usar o sistema
- As sequências de ações que podem ocorrer durante o uso do sistema são definidas

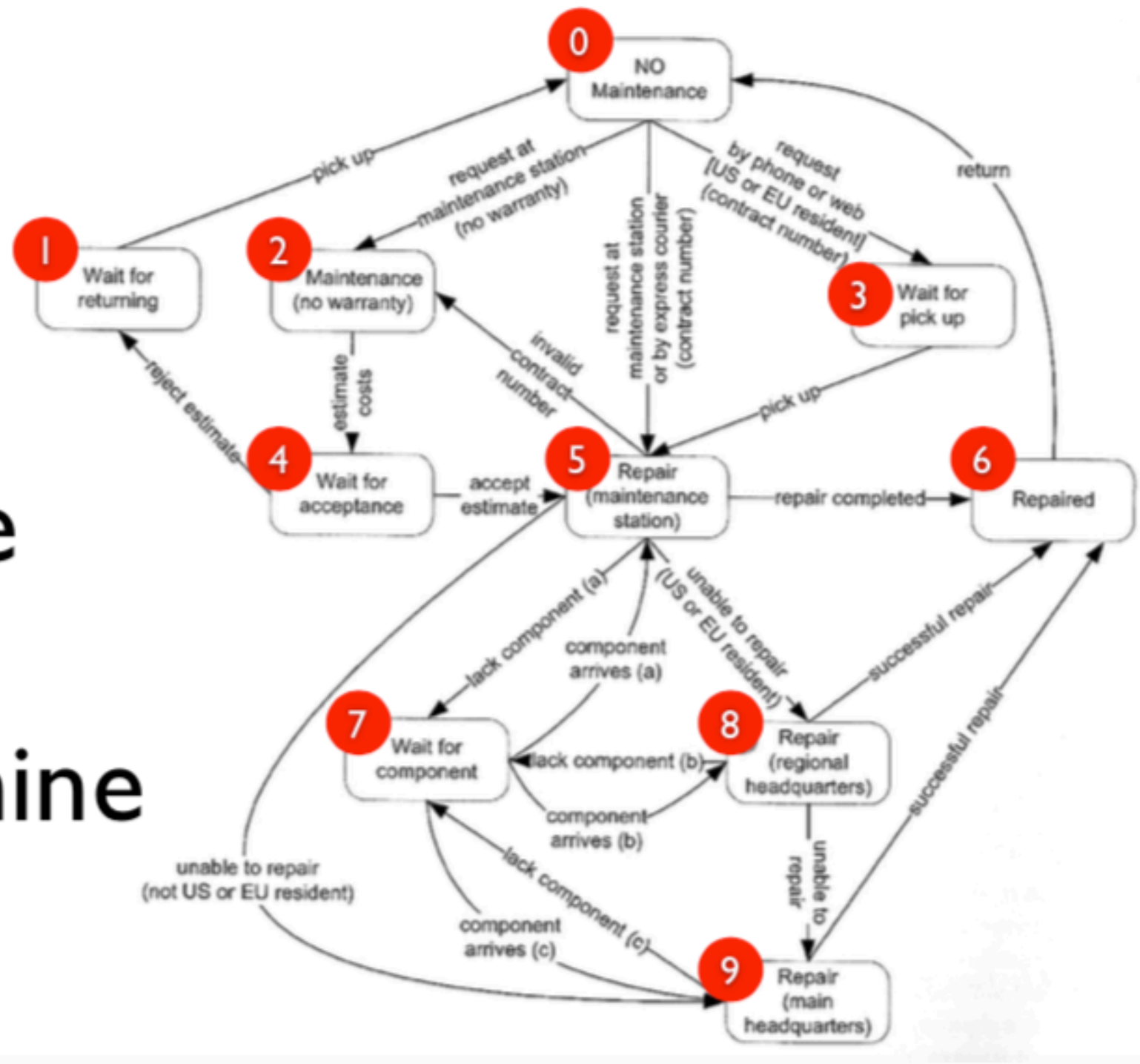
Teste Baseado em Modelo



- Tem um modelo formal que especifica o comportamento do software
- Modelos tipicamente vem com:
  - máquina de estados finitos e
  - estruturas de decisão

Teste Baseado em Modelo
Máquina de Estados Finitos

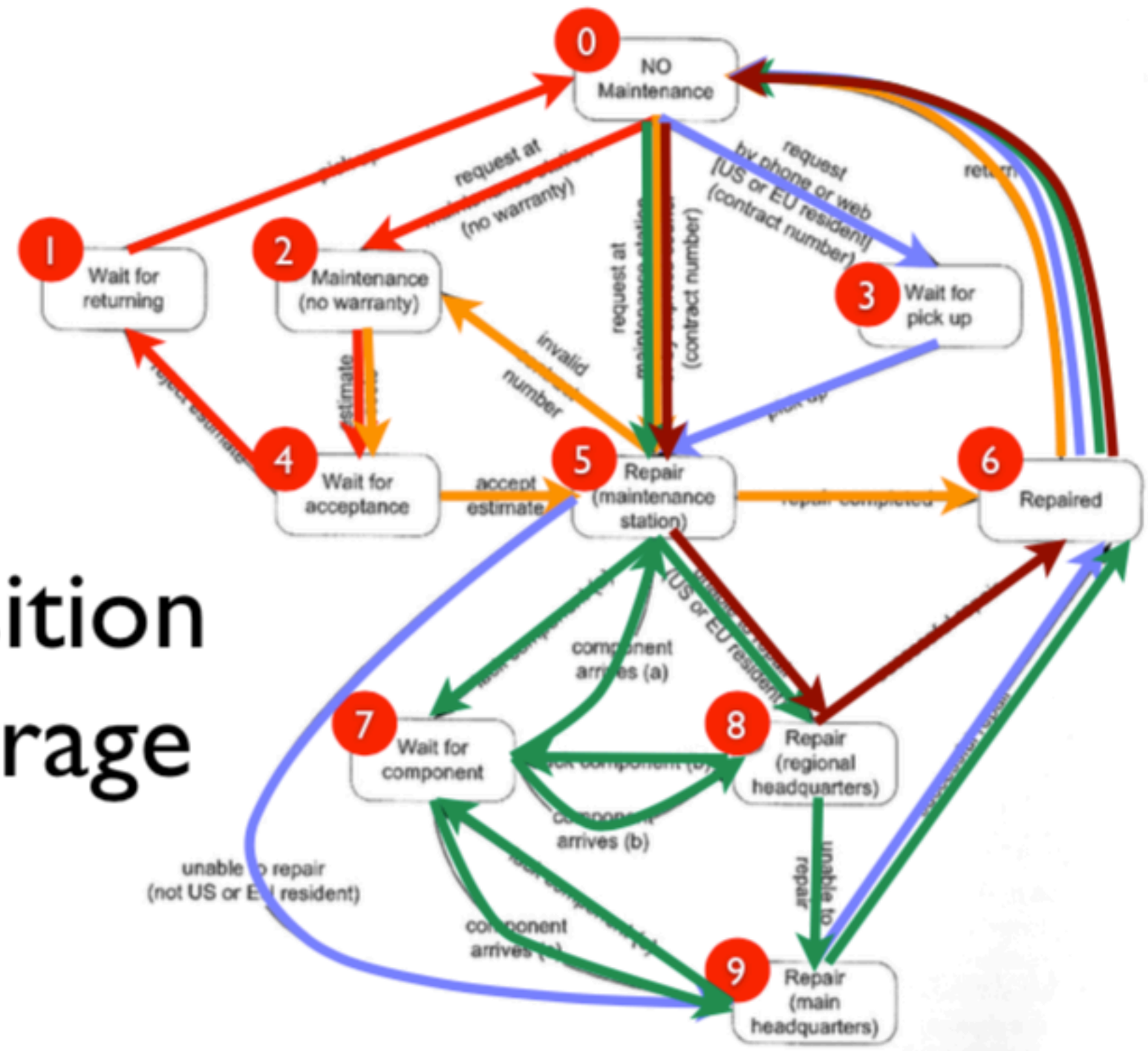
Finite State Machine



Teste Baseado em Modelo
Máquina de Estados Finitos

- **Critérios de cobertura**
  - Cobertura de caminhos
    - testes cobrem todos os caminhos —> impraticável devido ao numero exponencial de caminhos
  - Cobertura de estados
    - todos os nós são executados —> um critério mínimo
  - Cobertura de transição
    - toda transição é executada —> tipicamente, um bom critério de cobertura

Teste Baseado em Modelo
Máquina de Estados Finitos

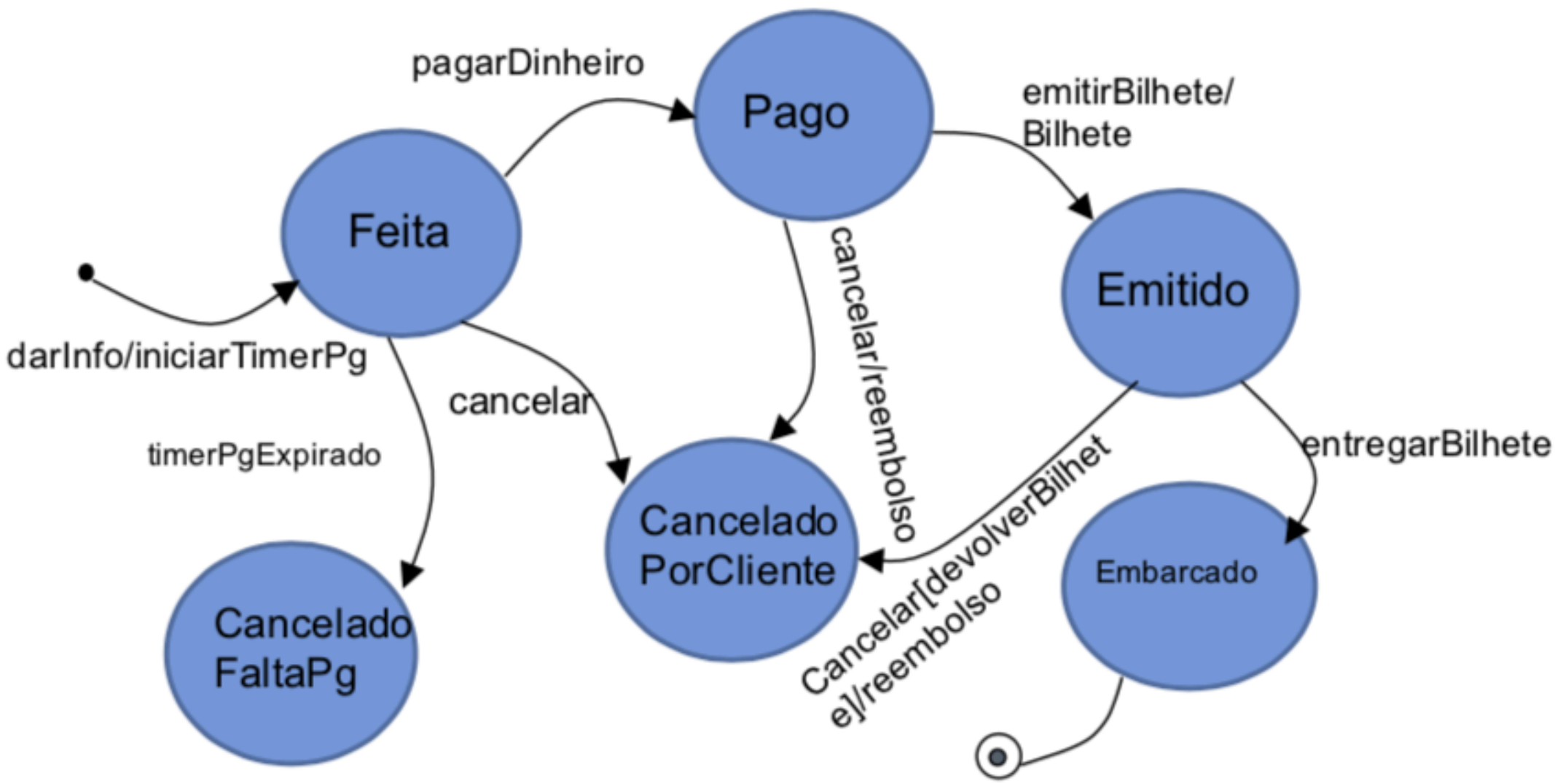


Transition Coverage

Cobertura de Transição —> Teste de Transição de Estados

- Um sistema pode exibir respostas diferentes dependendo da sua condição atual ou de estado anterior
  - Neste caso, o comportamento do sistema pode ser representado como um diagrama de transição de estados.
  - Permite ao testador visualizar o software em termos de estados, transições entre estados, as entradas ou eventos que disparam as mudanças de estado (transição) e as ações que podem resultar daquelas transições.

Cobertura de Transição —> Teste de Transição de Estados



Teste Baseado em Estados
Tabelas de Decisão

- Os estados do sistema, ou objetos sob teste, são isolados, identificáveis e finitos
- Uma tabela de estado exibe a relação entre estados e entradas, e pode destacar possíveis transições inválidas
- A tabela de decisão contém as condições que disparam as ações
- Excelente técnica para registrar regras de negócio complexas. Também servem como guia para criar casos de teste
- Grande vantagem da utilização da tabela de decisão é que ela cria combinações de condições que geralmente não foram exercitadas durante os testes
- Pode ser aplicada a todas as situações quando a execução do software depende de muitas decisões lógicas

Tabelas de Decisão

- Representam regras de negócio complexas baseadas em um conjunto de condições:

	Regra1	Regra2	...	Regra P
Condições				
Cond-1				
Cond-2				
...				
Cond-m				
Ações				
Ação-1				
Ação-2				
...				
Ação-n				

Tabelas de Decisão

- Condições de 1 a m, representam as condições de entrada
- Ações de 1 a n, representam ações a serem tomadas dependendo das combinações das várias condições
- Cada regra pode especificar uma ação exclusiva da regra, ou regras podem compartilhar ações:

	Regra1	Regra2	Regra3	Regra 4
Condições				
Casado ?	Sim	Sim	Não	Não
Bom Estudante?	Sim	Não	Sim	Não
Ações				
Desconto	*60*	*25*	*50*	*0*

Tabelas de Decisão

- Uma regra também pode possuir mais de uma ação; exclusiva ou compartilhada com outras regras

	Regra1	Regra2	Regra3	Regra 4
Condições				
Cond-1	Sim	Sim	Não	Não
Cond-2	Sim	Não	Sim	Não
Ações				
Ação-1	*Do X*	*Do Y*	*Do X*	*DO Z*
Ação-2	*Do-A*	*Do B*	*Do B*	*Do B*

- Valores das condições podem ser mais complexos

	Regra1	Regra2	Regra3	Regra 4
Condições				
Cond-1	0 - 1	1 - 10	10 - 100	100 - 100
Cond-2	< 5	5	6 ou 7	>7
Ações				
Ação-1	*Do X*	*Do Y*	*Do X*	*DO Z*
Ação-2	*Do-A*	*Do B*	*Do B*	*Do B*

Teste Baseado em Modelos/Estados

- Pode ser utilizado para modelar uma grande variedade de comportamentos, que serve de base para testar
  - Protocolos de rede (comunicação)
  - GUIs (sequências de interações)
  - Objetos (métodos e estados)

Tabelas de Decisão - outro exemplo

a,b,c form a triangle	F	T	T	T	T	T	T	T	T
a = b	–	T	T	T	T	F	F	F	F
a = c	–	T	T	F	F	T	T	F	F
b = c	–	T	F	T	F	T	F	T	F
Not a triangle	X								
Scalene									X
Isosceles					X		X	X	
Equilateral		X							
Impossible			X	X		X			

Critério de Cobertura

- **Básico**: testar cada coluna (valores *don't care* "-" podem ter valores arbitrários)

- **Composto**: testa todas as combinações —> requer 2^n testes p/ n condições —> impraticável

- **Modified condition decision (MC/DC)** —> condição/decisão modificada —> similar ao critério básico, mas modifica cada valor True/False pelo menos 1 vez

  - Um bom critério!

Modified Condition Decision Criterion (MC/DC)

A && B && C

Test case	A	B	C	Resultado
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Qual a combinação em que uma condição simples e independente afeta a saída/resultado da expressão?

Modified Condition Decision Criterion (MC/DC)

A && B && C

Test case	A	B	C	Resultado
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Qual a combinação em que uma condição simples e independente afeta a saída/resultado da expressão?

Modified Condition Decision Criterion (MC/DC)

A && **B** && C

Test case	A	B	C	Resultado
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Qual a combinação em que uma condição simples e independente afeta a saída/resultado da expressão?

Modified Condition Decision Criterion (MC/DC)

A && B && **C**

Test case	A	B	C	Resultado
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Qual a combinação em que uma condição simples e independente afeta a saída/resultado da expressão?

Modified Condition Decision Criterion (MC/DC)

A && B && C

Test case	A	B	C	Resultado
1	T	T	T	T
2	T	T	F	F
3	T	F	T	F
4	T	F	F	F
5	F	T	T	F
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

Qual a combinação em que uma condição simples e independente afeta a saída/resultado da expressão?

1	T	T	T	T
5	F	T	T	F
3	T	F	T	F
2	T	T	F	F

Critério mais forte!!

Regra de Pareto: 20/80

Aproximadamente 80% das falhas
são reveladas por 20% dos casos de teste!

Estudar

- Capítulo 14 - Livro texto 1
- Capítulo 3 - Livro texto 2

Referências

- **Livro texto1**: Pezze, M., Young, M. Teste e Análise de Software Processos, Princípios e Técnicas, editora Bookman, 2008.

- **Livro texto2**: Márcio Eduardo Delamaro, José Carlos Maldonado, Mario Jino. Introdução ao Teste de Software, editora Elsevier, 2016. 2ª Edição.

- Testing and Debugging Advanced Course - Summer September 2010. Saarland University. Gordon Fraser and Andreas Zeller. URL: https://www.st.cs.uni-saarland.de/edu/testingdebugging10/